IMPLEMENTATION OF A SCALABLE CLOUD-BASED SKILL GAP ANALYSIS AND PLACEMENT MANAGEMENT SYSTEM USING AWS AND TERRAFORM

Prepared in the partial fulfillment of the Summer Internship Program on AWS

AT

Under the guidance of

Mrs. Sumana Bethala, APSSDC

Mr. J Lovababu, APSSDC

Submitted by

C.H Maha Lakshmi Visistha, 24F41A0527, KUPPAM ENGINEERING COLLEGE, (Batch -1)

Vijay Kumar Raju, 24F41A0547, KUPPAM ENGINEERING COLLEGE, (Batch -1)

K Premalatha, 24F41A0565, KUPPAM ENGINEERING COLLEGE, (Batch -1)

K.H Jahnavi Padma Priya, 24F41A0560, KUPPAM ENGINEERING COLLEGE, (Batch -1)

Bhavana C, 24F41A0515, KUPPAM ENGINEERING COLLEGE, (Batch -1)

ACKNOWLEDGEMENT

I would like to express my heartfelt gratitude to all those who have contributed to the successful completion of my summer internship project at **Andhra Pradesh Skill Development Corporation (APSSDC)**. This opportunity has been an enriching and transformative experience for me, and I am truly thankful for the support, guidance, and encouragement I have received along the way.

First and foremost, I extend my sincere regards to **Mrs Sumana Bethala** and **Mr. J Lovababu**, my supervisors and mentors, for providing me with valuable insights, constant guidance, and unwavering support throughout the duration of the internship. Their expertise and encouragement have been instrumental in shaping the direction of this project.

I would like to thank the entire team at **Andhra Pradesh Skill Development Corporation (APSSDC)** for fostering a collaborative and innovative environment. The camaraderie, knowledge sharing, and feedback I received from my colleagues significantly contributed to the development and success of this project.

In conclusion, I am honoured to have been a part of this internship program, and I look forward to leveraging the skills and knowledge gained to contribute positively to future endeavours.
Thank you.

Sincerely,

C.H Maha Lakshmi Visistha, 24F41A0527, mahalakshmivisishtha@gmail.com
Vijay Kumar Raju, 24F41A0547, vijaykumarraju18@gmail.com
K Premalatha, 24F41A0565, premalathak666@gmail.com,
K.H Jahnavi Padma Priya, 24F41A0560, khjahnavipadmapriya23@gmail.com
Bhavana C, 24F41A0515, bhavanac1107@gmail.com

ABSTRACT

This project presents the implementation of a **Scalable Cloud-Based Skill Gap Analysis and Placement Management System Using AWS and Terraform**. The primary objective is to develop and deploy a cloud-based platform that helps students identify their skill gaps, manage placement activities, and simplify the recruitment process through a centralized web application.

The application is developed using **React.js** for the frontend and **Node.js with Express.js** for the backend. It provides features such as student registration and login, skill gap analysis, placement tracking, resume management, leaderboard generation, profile management, and an administrative dashboard. These modules enable students and administrators to efficiently manage placement-related information through a user-friendly interface.

The cloud infrastructure is deployed on **Amazon Web Services (AWS)** using **Terraform**, which automates the creation and management of cloud resources. **Amazon EC2** is used to host the application, while **Amazon S3** is used to securely store student resumes and documents. A secure network is established using **Amazon VPC**, along with public subnets, Internet Gateway, route tables, and security groups to control network traffic. The application also uses the **AWS LabInstanceProfile** to securely access AWS resources without storing credentials in the application.

The project also incorporates **CI/CD automation** to simplify application deployment and updates. By combining cloud computing, infrastructure automation, and modern web technologies, the system provides a scalable, secure, and efficient solution for placement management. This project demonstrates the practical use of AWS cloud services and Terraform in deploying real-world web applications while improving deployment efficiency, security, and maintainability.

TABLE OF CONTENTS

INTRODUCTION

The rapid advancement of cloud computing and web technologies has significantly transformed the way organizations develop, deploy, and manage software applications. Educational institutions are also adopting cloud-based solutions to improve academic management, streamline administrative tasks, and enhance the overall learning experience for students. Among these, placement management has become one of the most important areas where technology can improve efficiency, transparency, and decision-making.

In today's competitive job market, students are expected to possess strong technical skills, problem-solving abilities, and practical knowledge to secure employment opportunities. However, many students face difficulties in identifying their skill gaps, tracking their preparation progress, organizing resumes, and staying updated with placement activities. Similarly, placement administrators often find it challenging to monitor student performance, manage placement-related information, and provide personalized guidance using traditional methods. These limitations highlight the need for a centralized, automated, and scalable placement management solution.

The **Cloud-Based Skill Gap and Placement Management System** is designed to address these challenges by providing a comprehensive platform that assists students in improving their employability while enabling administrators to efficiently manage placement activities. The system allows students to register and log in securely, identify their skill gaps, track placement preparation, upload and manage resumes, monitor their overall progress, and view leaderboards that encourage continuous learning and healthy competition. The administrator dashboard provides centralized control for managing users, monitoring placement data, and maintaining the overall system.

The application follows a modern full-stack architecture. The frontend is developed using **React.js**, providing a responsive and interactive user interface, while the backend is implemented using **Node.js** and **Express.js**, which handle business logic, user authentication, and communication between different system components. The project structure is modular, making it easier to maintain, update, and extend in the future.

To ensure high availability, scalability, and secure deployment, the application is hosted on **Amazon Web Services (AWS)**. The backend server is deployed on **Amazon EC2**, which provides a reliable virtual computing environment capable of handling application workloads. Student resumes and uploaded documents are securely stored in **Amazon S3**, offering highly durable and

scalable object storage. The networking infrastructure is configured using **Amazon VPC**, **Public Subnet**, **Internet Gateway**, **Route Tables**, and **Security Groups**, ensuring secure communication between cloud resources while controlling network access.

A major aspect of this project is the adoption of **Infrastructure as Code (IaC)** using **Terraform**. Instead of manually creating cloud resources, Terraform automates the provisioning of AWS infrastructure through configuration files. This approach improves consistency, reduces deployment time, minimizes human errors, and allows the infrastructure to be recreated or modified efficiently whenever required. Infrastructure automation also follows modern DevOps practices and simplifies cloud resource management.

The project also incorporates deployment automation using **AWS CodePipeline**, **AWS CodeBuild**, and **AWS CodeDeploy**, enabling Continuous Integration and Continuous Deployment (CI/CD). These services automate the process of building, testing, and deploying application updates from the GitHub repository to the AWS environment. As a result, application updates can be delivered more quickly, reliably, and with minimal manual intervention.

Security is another important consideration in this project. Access to the application is controlled through secure user authentication, while AWS Security Groups regulate inbound and outbound network traffic. Cloud resources are isolated within a Virtual Private Cloud (VPC), and uploaded resumes are securely stored in Amazon S3 with appropriate access permissions. These measures help protect application data while ensuring secure communication between cloud services.

The primary objective of this project is to demonstrate how modern cloud technologies and infrastructure automation can be integrated to build a scalable, secure, and efficient placement management system. By leveraging AWS cloud services and Terraform, the project showcases best practices in cloud deployment, resource management, and application hosting. The developed system not only simplifies placement management but also provides students with an organized platform to enhance their skills, monitor their progress, and prepare effectively for future career opportunities.

Overall, this project serves as a practical implementation of cloud computing, full-stack web development, and DevOps principles, illustrating how modern technologies can be combined to develop real-world solutions for educational institutions. The proposed system offers improved scalability, reliability, security, and ease of management compared to traditional deployment approaches, making it a suitable solution for modern placement management requirements.

METHODOLOGY

The development of the Cloud-Based Skill Gap and Placement Management System followed a systematic methodology to design, develop, deploy, and test the application efficiently. The project was carried out in multiple phases, including requirements gathering, system design, implementation, infrastructure provisioning, testing, deployment, and iterative refinement. Each phase contributed to building a scalable, secure, and cloud-based placement management solution.

2.1 Requirements Gathering

The project began by analyzing the requirements of students and placement administrators. The primary objective was to develop a centralized platform that provides **user authentication**, **skill gap analysis**, placement tracking, resume management, and administrative functionalities. The system was also planned for deployment on the **AWS Cloud** using **Terraform** for infrastructure automation.

2.2 System Design

Based on the identified requirements, a full-stack architecture was designed. The frontend was developed using **React.js**, while the backend was implemented using **Node.js** and **Express.js**. The cloud infrastructure was designed using **Amazon EC2** for application hosting and **Amazon S3** for secure resume storage. Networking components such as VPC, subnets, route tables, and security groups were configured to provide secure communication between cloud resources.

2.3 Implementation

The application was developed by integrating the frontend and backend modules. Features including user registration, login, dashboard, skill gap analysis, placement tracker, resume management, leaderboard, profile management, and administrator dashboard were implemented. Communication between the frontend and backend was achieved through **REST APIs**, while uploaded resumes were securely stored in **Amazon S3**.

2.4 Infrastructure Provisioning and Deployment

The cloud infrastructure was provisioned using **Terraform**, which automatically created the required AWS resources. The application was deployed on an **Amazon EC2** instance, with **Nginx** configured as the web server and **PM2** used to manage the Node.js application. Deployment automation was achieved using **AWS CodePipeline**, **CodeBuild**, and **CodeDeploy**.

2.5 Testing

The application was tested to verify the functionality of all modules. **Functional Testing** was performed to validate authentication, placement tracking, resume uploads, and administrator features. **Integration Testing** ensured smooth communication between the frontend, backend, and AWS services. The Terraform configuration was also validated to ensure successful infrastructure deployment.

2.6 Iterative Refinement

The application was continuously improved based on testing results and mentor feedback. Enhancements were made to improve the user interface, optimize backend performance, strengthen deployment, and improve the overall reliability of the application.

TECHNOLOGY STACK

The **Placement Management System** was developed using modern web technologies and Amazon Web Services (AWS) to provide a scalable, secure, and cloud-based solution. The technologies and services used in this project are described below.

3.1 Cloud Platform

- **Amazon Web Services (AWS):** Used to host and manage the cloud infrastructure for the application.

3.2 AWS Services

- **Amazon EC2 (Elastic Compute Cloud):** Hosts the Placement Management System.
- **Amazon S3 (Simple Storage Service):** Stores student resumes and uploaded documents.

- **Amazon VPC (Virtual Private Cloud):** Provides a secure virtual network for cloud resources.
- **Public Subnet, Internet Gateway, and Route Table:** Enable secure internet connectivity for the EC2 instance.
- **Security Groups:** Control inbound and outbound network traffic.
- **AWS IAM (Identity and Access Management):** Manages secure access and permissions for AWS resources.

3.3 Infrastructure as Code

- **Terraform:** Automates the provisioning and management of AWS infrastructure.

3.4 Frontend Technologies

- **React.js:** Develops the user interface.
- **Vite:** Frontend build tool.
- **HTML5, CSS3, and JavaScript:** Used for designing responsive web pages and implementing client-side functionality.

3.5 Backend Technologies

- **Node.js:** Executes the backend application.
- **Express.js:** Develops RESTful APIs and handles server-side requests.

3.6 Database

- **SQLite:** Stores user information, skill details, placement records, and application data.

3.7 DevOps & Deployment

- **AWS CodePipeline:** Automates the CI/CD workflow.
- **AWS CodeBuild:** Builds and tests the application.
- **AWS CodeDeploy:** Deploys the application to the EC2 instance.
- **Git & GitHub:** Used for version control and source code management.

3.8 Server Management

- **Nginx:** Serves the frontend and acts as a reverse proxy.

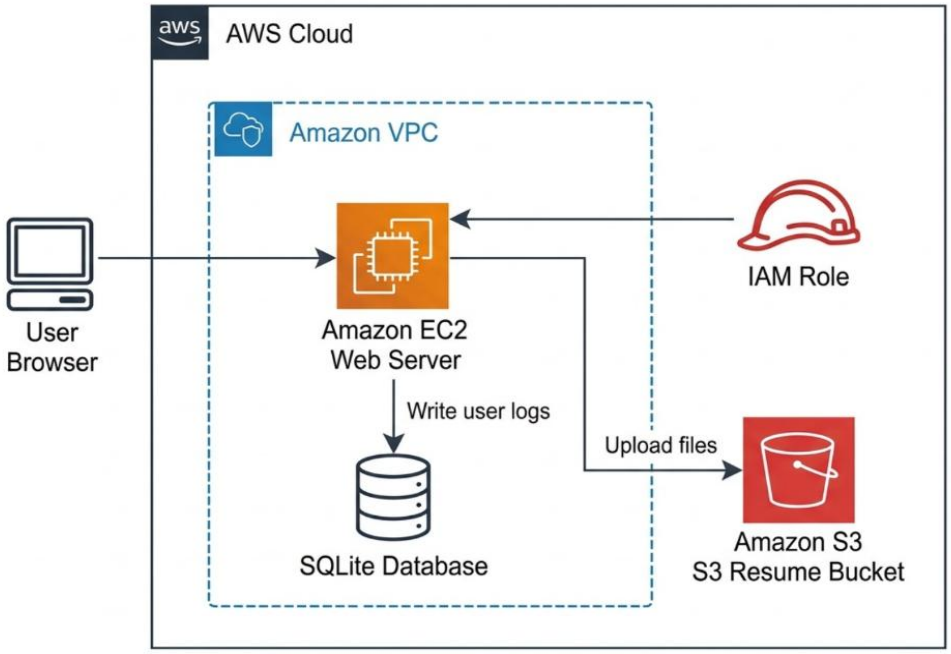
- **PM2:** Manages the Node.js backend application.

SYSTEM DESIGN

The **Placement Management System** is designed as a cloud-based web application that enables students and administrators to manage placement-related activities efficiently. The system follows a three-tier architecture consisting of the presentation layer, application layer, and cloud infrastructure layer. AWS services are used to provide secure, scalable, and reliable hosting, while Terraform automates the deployment of the infrastructure.

Overall System Architecture

The application follows a client-server architecture where users access the system through a web browser. The frontend communicates with the backend through REST APIs, and the backend stores application data in the database while managing resume uploads to Amazon S3.



Frontend Layer

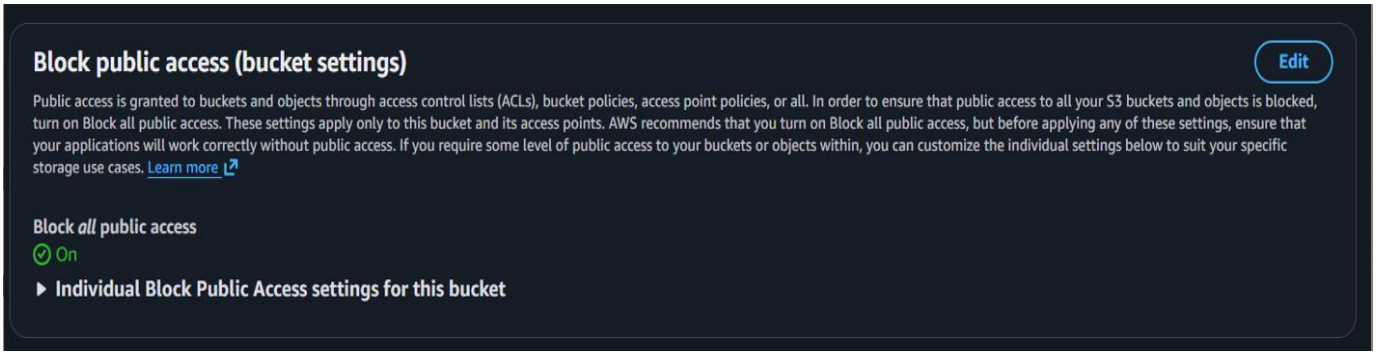
The frontend is developed using **React.js** and **Vite**. It provides an interactive interface for students and administrators to register, log in, manage profiles, upload resumes, view skill gaps, and track placement activities.

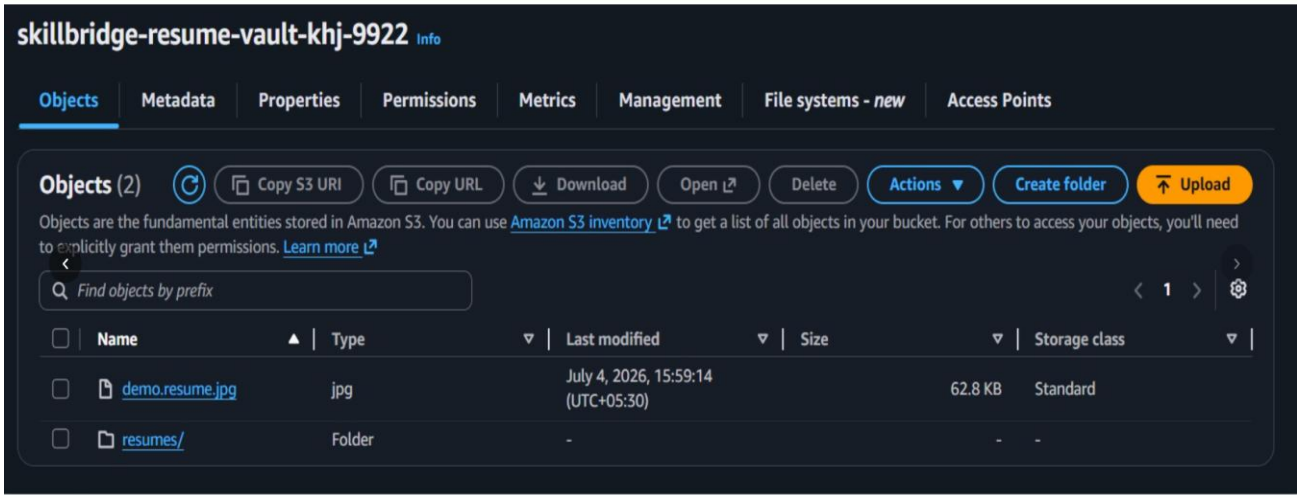
Backend Layer

The backend is developed using **Node.js** and **Express.js**. It processes user requests, implements business logic, authenticates users, manages placement records, and communicates with the database and Amazon S3.

The backend consists of modules such as:

- Authentication
- Skill Gap Analysis
- Placement Tracking
- Resume Management
- Leaderboard
- Admin Management





Database Layer

The system uses **SQLite** to store application data including:

- Student Information
- Login Credentials
- Skill Records
- Placement Details
- Resume Information

The database ensures efficient retrieval and secure storage of application data.

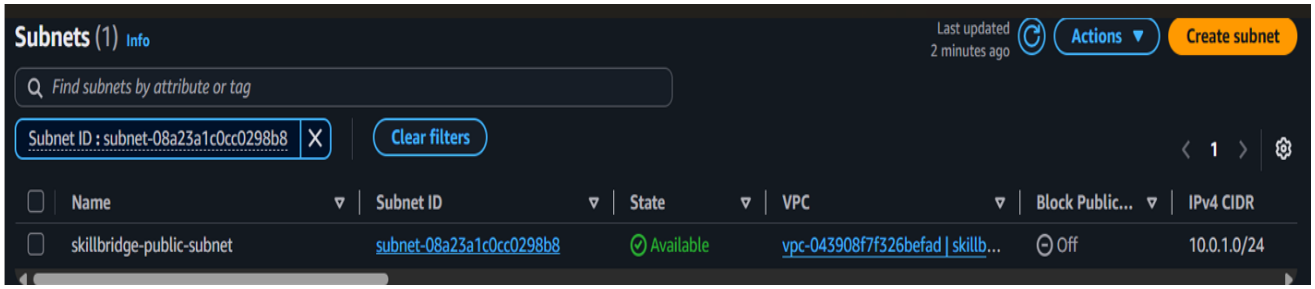
Cloud Infrastructure (AWS)

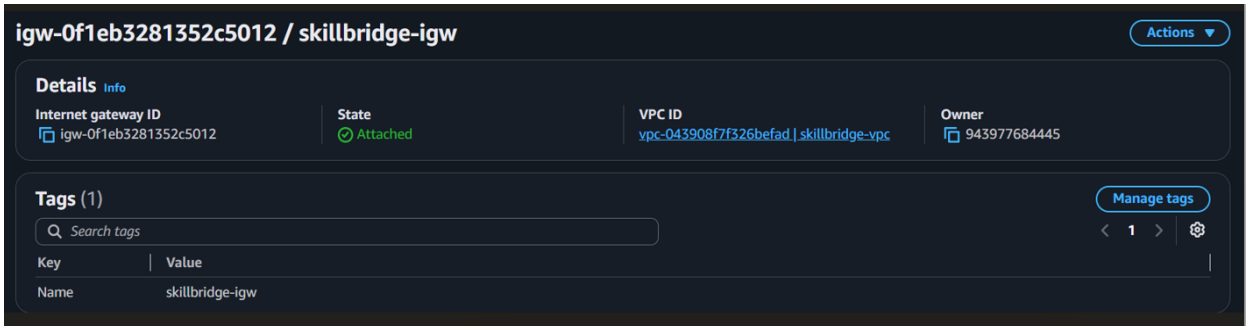
The application is deployed on **Amazon EC2** within an **Amazon VPC**. The infrastructure is created automatically using **Terraform**.

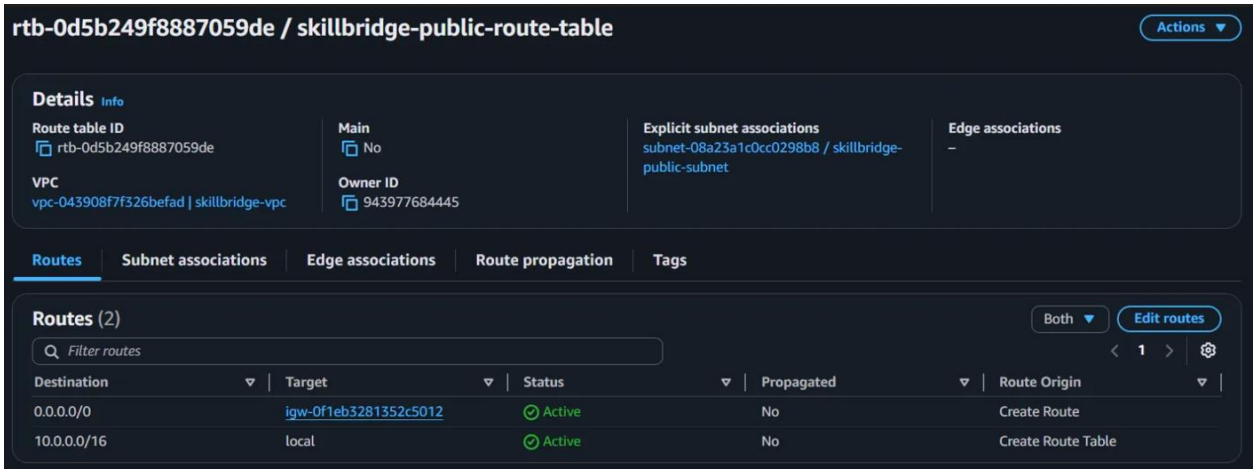
The deployed resources include:

- Amazon EC2
- Amazon S3

- Amazon VPC
- Public Subnet
- Internet Gateway
- Route Table
- Security Group
- IAM Role / LabInstanceProfile







Infrastructure Automation Using Terraform

Terraform is used to provision all AWS resources automatically. Infrastructure as Code (IaC) improves deployment consistency, reduces manual configuration, and simplifies infrastructure management.

The Terraform configuration creates:

- VPC
- Subnet
- Internet Gateway
- Route Table
- Security Group
- EC2 Instance
- S3 Bucket

```
# 2. Security Group (Web & SSH access)
resource "aws_security_group" "web_sg" {
  name        = "skillbridge-web-security-group"
  description = "Enable inbound HTTP, SSH, and Node API traffic"
  vpc_id      = aws_vpc.main_vpc.id

  # SSH
  ingress {
    from_port   = 22
    to_port     = 22
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # HTTP
  ingress {
    from_port   = 80
    to_port     = 80
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  # Express Backend API Port
  ingress {
    from_port   = 5000
    to_port     = 5000
    protocol    = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }
```

CI/CD Pipeline

The project uses AWS CI/CD services to automate application deployment.

The deployment workflow includes:

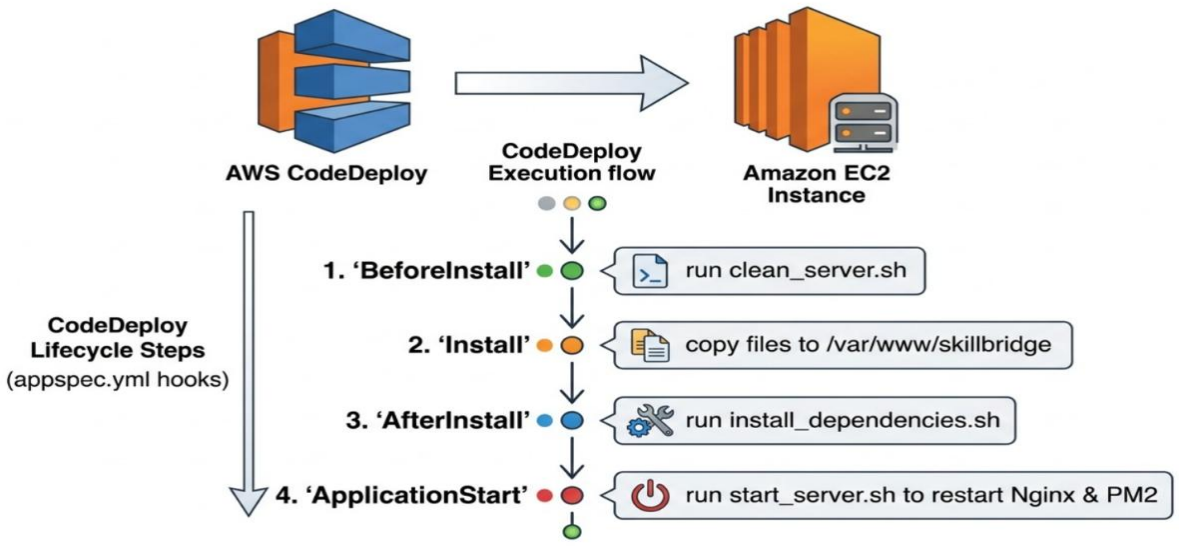
GitHub → AWS CodePipeline → CodeBuild → CodeDeploy → EC2

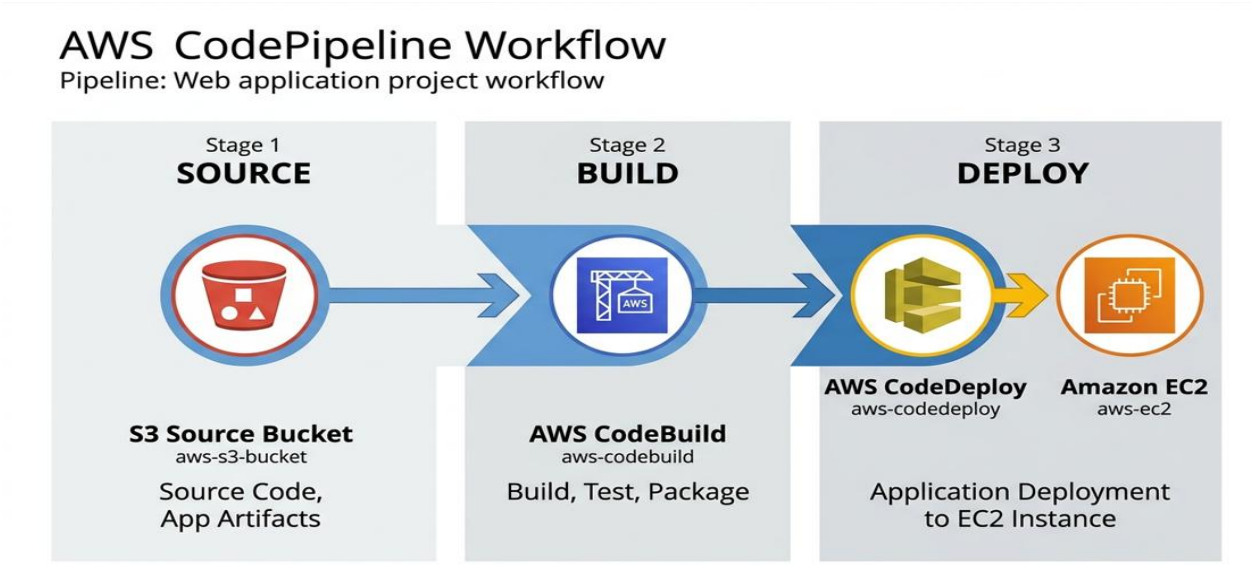
This ensures that every code update is automatically built and deployed to the EC2 instance.

Server Management

The application is hosted on an EC2 instance where:

- **Nginx** serves the frontend.
- **PM2** manages the Node.js backend.
- Security Groups allow HTTP and SSH access.



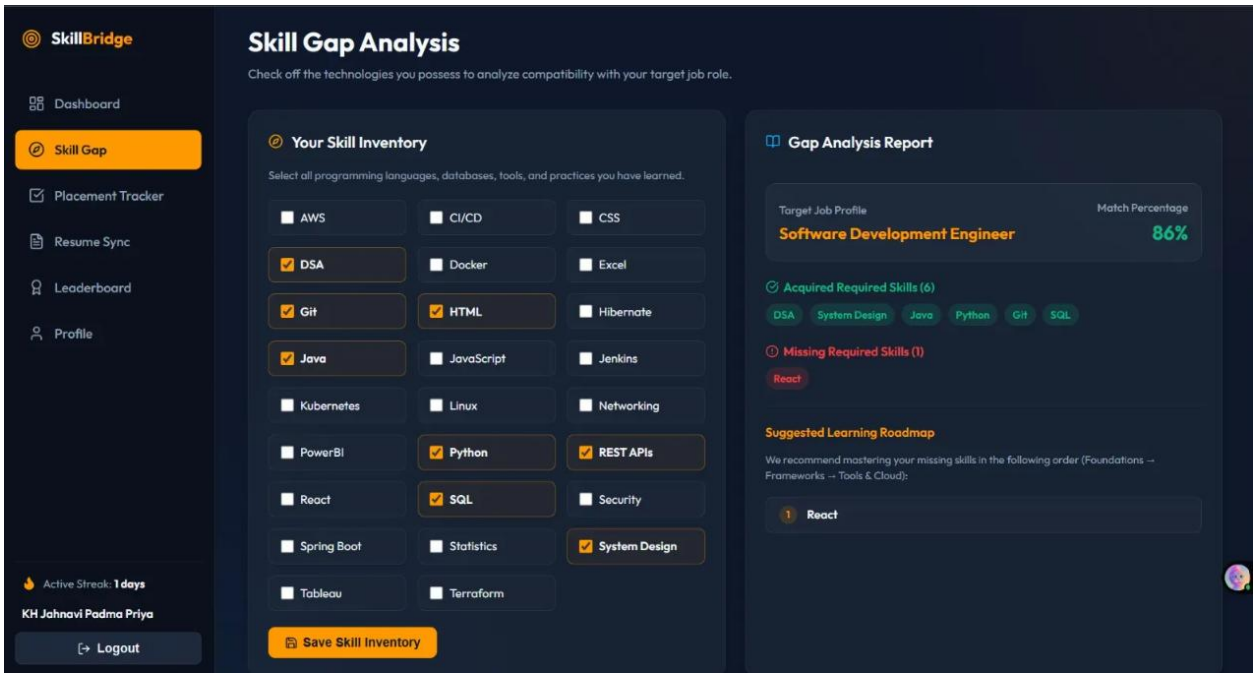


IMPLEMENTAION

The **Placement Management System** was implemented as a cloud-based web application using modern web technologies and Amazon Web Services (AWS). The system was developed by integrating a React-based frontend, a Node.js and Express.js backend, SQLite database, and AWS cloud infrastructure. Terraform was used to automate the provisioning of cloud resources, while AWS CI/CD services enabled automated deployment.

5.1 Frontend Implementation

The frontend of the application was developed using **React.js** and **Vite** to provide a responsive and user-friendly interface. Multiple web pages were created for student registration, login, dashboard, skill gap analysis, placement tracking, resume management, leaderboard, profile management, and administrator functions. The frontend communicates with the backend through RESTful APIs to exchange data.



5.2 Backend Implementation

The backend was implemented using **Node.js** and **Express.js**. It handles user authentication, manages placement records, processes skill information, handles resume uploads, and performs administrative operations. The backend also provides REST APIs that enable communication between the frontend and the database.

5.3 Database Implementation

The application uses **SQLite** as the database management system to store user information, placement records, skill details, resume information, and other application data. The database supports efficient CRUD (Create, Read, Update, Delete) operations, ensuring reliable data storage and retrieval.

5.4 Cloud Infrastructure Implementation

The cloud infrastructure was deployed on **Amazon Web Services (AWS)**. **Terraform** was used as an Infrastructure as Code (IaC) tool to automatically provision resources such as **Amazon EC2**, **Amazon S3**, **Amazon VPC**, **Public Subnet**, **Internet Gateway**, **Route Table**, and **Security Groups**. This approach simplifies infrastructure management and ensures consistent deployments.

5.5 Application Deployment

The application was deployed on an **Amazon EC2** instance. The React frontend was served using **Nginx**, while the Node.js backend was managed using **PM2** to ensure continuous execution. Student resumes and uploaded documents were securely stored in an **Amazon S3** bucket. This deployment architecture provides high availability and reliable application hosting.

5.6 CI/CD Implementation

A Continuous Integration and Continuous Deployment (**CI/CD**) pipeline was implemented using **AWS CodePipeline**, **AWS CodeBuild**, and **AWS CodeDeploy**. The pipeline automatically retrieves the latest source code from GitHub, builds the application, and deploys it to the EC2 instance. This automation reduces manual effort and ensures faster, consistent deployments.

5.7 Security Implementation

Security was implemented using **Amazon VPC** and **Security Groups** to protect the application infrastructure. Network access was controlled by allowing only the required ports for HTTP, SSH, and backend communication. **AWS IAM** was used to securely manage permissions for AWS resources, ensuring controlled and authorized access

RESULT

The implementation of the **Placement Management System** successfully achieved the project's objectives of providing a cloud-based platform for managing student placement activities, skill gap analysis, resume management, and placement tracking. The system was tested under various scenarios to evaluate its functionality, performance, security, and deployment on AWS. The following results demonstrate the effectiveness of the developed application.

6.1. User Authentication and Access Control

The system provides secure user registration and login functionality for students and administrators. Authentication ensures that only authorized users can access their respective dashboards and features. Role-based access control allows administrators to manage system data while students can access their personal placement information.

6.2. Skill Gap Analysis and Placement Tracking

The application successfully identifies students' skill gaps and allows them to monitor their placement progress. Students can update their profiles, track placement opportunities, and view relevant information through an interactive dashboard. This helps users understand the skills required for improving their placement readiness.

6.3. Resume Management

The Resume Management module allows students to upload and manage their resumes efficiently. Resume files are securely stored in **Amazon S3**, ensuring reliable storage and easy retrieval whenever required.

6.4. Cloud Deployment and Infrastructure

The application was successfully deployed on **Amazon EC2** using cloud infrastructure created through **Terraform**. AWS resources including **Amazon VPC**, **Public Subnet**, **Internet Gateway**, **Route Table**, **Security Groups**, and **Amazon S3** were provisioned automatically, demonstrating the effectiveness of Infrastructure as Code (IaC).

6.5. Automated Deployment

A Continuous Integration and Continuous Deployment (**CI/CD**) pipeline was implemented using **AWS CodePipeline**, **AWS CodeBuild**, and **AWS CodeDeploy**. The deployment process automatically fetched the latest code from GitHub, built the application, and deployed it to the EC2 instance, reducing manual effort and improving deployment efficiency.

6.6. Security and System Performance

The application was secured using **AWS Security Groups** and **IAM** permissions to control access to cloud resources. The system was tested under multiple user interactions, including registration, login, resume upload, and placement tracking. The application performed reliably with smooth navigation and stable response times.

Testing scenarios included:

- User Registration and Login
- Skill Gap Analysis
- Resume Upload and Storage
- Placement Tracking
- Dashboard Navigation
- AWS Infrastructure Deployment
- CI/CD Pipeline Execution

All test scenarios were executed successfully without any major issues.

CONCLUSION

The **Placement Management System** was successfully designed, developed, and deployed as a cloud-based web application using modern web technologies and Amazon Web Services (AWS). The project effectively integrates **React.js**, **Node.js**, **SQLite**, and **Terraform** with AWS services such as **Amazon EC2**, **Amazon S3**, **Amazon VPC**, **Security Groups**, and **IAM** to provide a secure, scalable, and efficient platform for managing placement activities.

The application enables students to register, analyze their skill gaps, upload resumes, track placement progress, and access relevant placement information through an intuitive interface. Administrators can efficiently manage student records and placement-related data. The use of **Terraform** simplified infrastructure provisioning through Infrastructure as Code (IaC), while the implementation of a **CI/CD pipeline** using AWS CodePipeline, CodeBuild, and CodeDeploy automated the deployment process, reducing manual effort and improving system reliability.

The project demonstrates the practical application of **Cloud Computing**, **Infrastructure as Code**, and **DevOps** practices in developing a real-world placement management solution. It provides a strong foundation for future enhancements such as AI-based skill recommendations, interview scheduling, company portals, email notifications, real-time analytics dashboards, and domain-based website deployment.

Overall, the project successfully met its objectives by delivering a reliable, scalable, and cloud-enabled Placement Management System that enhances the efficiency of placement management while providing valuable practical experience in full-stack development and AWS cloud technologies.